

Topics on Machine Learning and its Applications

Data Science Project requirements

The objective of the Data Science project is to enable students to practice and develop Machine Learning techniques by applying them to a real-world or academic problem of their own interest. The project is carried out individually, and each student is required to independently design and implement a complete Data Science pipeline, including data preprocessing, model training, and result analysis. This individual work aims to foster hands-on experience with real datasets, encourage autonomous methodological decision-making, and promote a critical interpretation of the obtained results.

The project accounts for 40% of the final course grade and will be evaluated based on the correctness, completeness, and clarity of the approach, the depth of the analysis, and the effectiveness of the final presentation. It should address the following components:

1. Problem definition and dataset selection

- Clearly define the problem statement by explaining its context and relevance.
- Justify the choice of dataset and discuss its potential to address the problem.
- Provide basic exploratory data analysis (EDA), including:
 - Summary statistics (e.g., mean, median, standard deviation).
 - Visualizations to understand distributions and relationships (e.g., histograms, scatter plots, correlation heatmaps).
 - Identification of possible biases or limitations in the dataset.

2. Data preprocessing and feature engineering

The dataset must be cleaned and prepared to ensure high-quality input for the models. This includes:

- Handling missing data: Apply appropriate strategies (e.g., imputation, deletion).
- Outlier detection and treatment: Use statistical or visualization methods to detect and handle outliers.
- Feature engineering:
 - Transform numerical features if necessary (e.g., log transformation, binning).
 - Encode categorical features (e.g., one-hot encoding, label encoding, ordinal encoding).

- Create new meaningful features (if relevant).
- Drop irrelevant features (if applicable).

3. Class imbalance handling (if applicable)

If the dataset presents an imbalance in class distribution, students must:

- Analyze class distribution and discuss its potential impact on model performance.
- Justify whether imbalance correction is necessary.
- Apply and compare one of the following methods if needed:
 - Resampling techniques (oversampling, undersampling).
 - Synthetic data generation (e.g., SMOTE or similar techniques).

4. Model selection and justification

Students must experiment with and compare at least three different machine learning algorithms, covering the following aspects:

- Justify the choice of algorithm based on the problem type (e.g., classification, regression).
- Implement and train the models, ensuring proper hyperparameter settings.
- If applicable, perform hyperparameter tuning using techniques such as grid search or random search.
- Discuss model complexity and interpretability, considering trade-offs between predictive performance and explainability.

5. Model evaluation and interpretation

Model performance must be thoroughly evaluated:

- Data Splitting: Ensure proper separation of training, validation, and test sets, and justify the method used.
- For classification problems: use the metrics Accuracy, Precision, Recall, F1-score, and ROC-AUC.
- For regression problems: use the metrics Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 , Mean Absolute Error (MAE).
- Discuss the significance of each metric in the context of the problem.
- Provide a critical analysis of model strengths, weaknesses, and potential improvements.

6. Reproducibility and code quality

Each project must be developed in Python and delivered as a single Jupyter Notebook file to ensure full reproducibility. The notebook must be fully executable from start to end, allowing professors to run all cells without errors. All code must be presented together with its corresponding execution output, and the analysis should be structured using small, well-defined code cells, avoiding excessively large code blocks. This organization is intended to improve readability, transparency, and the traceability of results.

The following requirements must be met:

- Code Structure and documentation:
 - Use modular and well-organized code.
 - Include comments and Markdown explanations for each step.
- Dependencies and environment:
 - Provide a requirements.txt file.

7. Final report and presentation

Each project must be summarized in a structured report and presented.

- Final report (within the Jupyter Notebook as Markdown)
 - Introduction: Problem definition and objectives.
 - Data exploration and preprocessing: Description of dataset issues and how they were handled.
 - Feature engineering: Explanation of feature selection and transformations.
 - Modeling and evaluation: Justification of model choices and detailed evaluation results.
 - Conclusion: Summary of findings, limitations, and potential future improvements.
- Presentation
 - The presentation must highlight the key aspects of the project in a concise manner.
 - Use clear visualizations to explain results.
 - Each student will present their work in 20 minutes, followed by a 10-minute discussion with the professor, where they must answer questions about their methodology, results, and limitations.